

My Project

1

Generated by Doxygen 1.9.7

1 Namespace Index	1
1.1 Namespace List	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Namespace Documentation	7
4.1 MLWENO Namespace Reference	7
4.1.1 Detailed Description	7
4.2 tensorProductPoly Namespace Reference	7
5 Class Documentation	9
5.1 tensorProductPoly::basePolynomial Class Reference	9
5.1.1 Constructor & Destructor Documentation	9
5.1.1.1 basePolynomial() [1/3]	9
5.1.1.2 basePolynomial() [2/3]	10
5.1.1.3 basePolynomial() [3/3]	10
5.1.1.4 ~basePolynomial()	10
5.1.2 Member Function Documentation	10
5.1.2.1 der() [1/2]	10
5.1.2.2 der() [2/2]	10
5.1.2.3 eval() [1/2]	10
5.1.2.4 eval() [2/2]	11
5.1.2.5 getCoef() [1/2]	11
5.1.2.6 getCoef() [2/2]	11
5.1.2.7 getI()	11
5.1.2.8 getJ()	11
5.1.2.9 getSize()	11
5.1.2.10 operator>() [1/2]	11
5.1.2.11 operator>() [2/2]	12
5.1.2.12 printCoef()	12
5.1.2.13 setCoef()	12
5.1.2.14 setMaxDegree()	12
5.2 MeshInfo Struct Reference	12
5.2.1 Member Data Documentation	13
5.2.1.1 cellGhostLayerSize	13
5.2.1.2 dim	13
5.2.1.3 edgeCorner	14
5.2.1.4 faceCorner	14
5.2.1.5 lmesh	14
5.2.1.6 localVals	14

5.2.1.7 MPIglobalCellSize	14
5.2.1.8 MPIglobalHoriEdgeSize	14
5.2.1.9 MPIglobalVertEdgeSize	14
5.2.1.10 MPIglobalVertexSize	15
5.2.1.11 MPIlocalCellSize	15
5.2.1.12 MPIlocalCellSizeFull	15
5.2.1.13 MPIlocalCellStart	15
5.2.1.14 MPIlocalHoriEdgeSize	15
5.2.1.15 MPIlocalVertEdgeSize	15
5.2.1.16 MPIlocalVertexSize	16
5.2.1.17 MPIlocalVertexSizeFull	16
5.2.1.18 MPIlocalVertexStart	16
5.2.1.19 vertexGhostLayerSize	16
5.2.1.20 volumeCorner	16
5.3 MLWENO::multiLevelReconstruction Class Reference	16
5.3.1 Constructor & Destructor Documentation	17
5.3.1.1 multiLevelReconstruction() [1/3]	17
5.3.1.2 multiLevelReconstruction() [2/3]	17
5.3.1.3 multiLevelReconstruction() [3/3]	18
5.3.1.4 ~multiLevelReconstruction()	18
5.3.2 Member Function Documentation	18
5.3.2.1 AddLevel() [1/3]	18
5.3.2.2 AddLevel() [2/3]	18
5.3.2.3 AddLevel() [3/3]	19
5.3.2.4 Clear()	19
5.3.2.5 GetInfo()	19
5.3.2.6 ModifyReconstMethod() [1/2]	19
5.3.2.7 ModifyReconstMethod() [2/2]	19
5.3.2.8 PrintBoundaryLayer()	20
5.3.2.9 PrintNonLinearWgts()	20
5.3.2.10 PrintSmoothnessIndicator()	20
5.3.2.11 SelectWenoReconstLevel()	20
5.3.2.12 SeparateBoundaryLayer() [1/2]	20
5.3.2.13 SeparateBoundaryLayer() [2/2]	20
5.3.2.14 UpdateNonLinearWgts()	21
5.4 MLWENO::singleLevelReconstruction Class Reference	21
5.4.1 Constructor & Destructor Documentation	21
5.4.1.1 singleLevelReconstruction() [1/2]	22
5.4.1.2 singleLevelReconstruction() [2/2]	22
5.4.1.3 ~singleLevelReconstruction()	22
5.4.2 Member Function Documentation	22
5.4.2.1 CalculateSmoothnessIndic()	22

5.4.2.2 CheckExist()	22
5.4.2.3 CheckStencilPolynomials()	23
5.4.2.4 CheckStencils()	23
5.4.2.5 CreateStencilPolynomials()	23
5.4.2.6 GetScale() [1/2]	23
5.4.2.7 GetScale() [2/2]	23
5.4.2.8 GetSizeX()	23
5.4.2.9 GetSizeY()	24
5.4.2.10 GetSmoothnessIndic()	24
5.4.2.11 PrintSmoothnessIndicator()	24
5.4.2.12 UpdateSmoothnessIndic()	24
5.5 stencil< T > Class Template Reference	24
5.5.1 Constructor & Destructor Documentation	25
5.5.1.1 stencil() [1/4]	25
5.5.1.2 stencil() [2/4]	25
5.5.1.3 stencil() [3/4]	26
5.5.1.4 stencil() [4/4]	26
5.5.1.5 ~stencil()	26
5.5.2 Member Function Documentation	26
5.5.2.1 clearPtr()	26
5.5.2.2 CreateStencil()	26
5.5.2.3 getI()	26
5.5.2.4 getJ()	27
5.5.2.5 getK()	27
5.5.2.6 getSize()	27
5.5.2.7 GetStencil()	27
5.5.2.8 operator() [1/6]	27
5.5.2.9 operator() [2/6]	27
5.5.2.10 operator() [3/6]	28
5.5.2.11 operator() [4/6]	28
5.5.2.12 operator() [5/6]	28
5.5.2.13 operator() [6/6]	28
5.5.2.14 setI()	28
5.5.2.15 setJ()	29
5.5.2.16 setK()	29
5.5.2.17 SetStencil() [1/3]	29
5.5.2.18 SetStencil() [2/3]	29
5.5.2.19 SetStencil() [3/3]	29
5.6 tensorProductPoly::stencilPolynomial Class Reference	29
5.6.1 Constructor & Destructor Documentation	30
5.6.1.1 stencilPolynomial() [1/3]	30
5.6.1.2 stencilPolynomial() [2/3]	30

5.6.1.3 stencilPolynomial() [3/3]	30
5.6.1.4 ~stencilPolynomial()	31
5.6.2 Member Function Documentation	31
5.6.2.1 eval() [1/2]	31
5.6.2.2 eval() [2/2]	31
5.6.2.3 GetOrderX()	31
5.6.2.4 GetOrderY()	31
5.6.2.5 GetScale()	31
5.6.2.6 GetSmoothIndic()	31
5.6.2.7 operator() [1/2]	32
5.6.2.8 operator() [2/2]	32
5.6.2.9 printCoef() [1/2]	32
5.6.2.10 printCoef() [2/2]	32
5.6.2.11 SetCollapsePolyn()	32
5.6.2.12 SetStencilPolynomials()	32
5.6.2.13 SetUpScale() [1/2]	33
5.6.2.14 SetUpScale() [2/2]	33
6 File Documentation	35
6.1 /home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp File Reference	35
6.2 /home/renpo/Research/MantleSolver/src/mlweno/polynomial.h File Reference	35
6.3 polynomial.h	36
6.4 /home/renpo/Research/MantleSolver/src/mlweno/reconstruction.cpp File Reference	37
6.5 /home/renpo/Research/MantleSolver/src/mlweno/reconstruction.h File Reference	37
6.6 reconstruction.h	38
6.7 /home/renpo/Research/MantleSolver/src/mlweno/stencil.h File Reference	39
6.7.1 Enumeration Type Documentation	40
6.7.1.1 stencilType	40
6.8 stencil.h	40
6.9 /home/renpo/Research/MantleSolver/src/mlweno/util.cpp File Reference	41
6.9.1 Function Documentation	42
6.9.1.1 AssignValuesMeshInfo()	42
6.9.1.2 basePoly()	42
6.9.1.3 Bend() [1/2]	42
6.9.1.4 Bend() [2/2]	42
6.9.1.5 constFunc() [1/4]	42
6.9.1.6 constFunc() [2/4]	43
6.9.1.7 constFunc() [3/4]	43
6.9.1.8 constFunc() [4/4]	43
6.9.1.9 factorial() [1/2]	43
6.9.1.10 factorial() [2/2]	43
6.9.1.11 FlatIndic() [1/4]	43

6.9.1.12 FlatIndic() [2/4]	44
6.9.1.13 FlatIndic() [3/4]	44
6.9.1.14 FlatIndic() [4/4]	44
6.9.1.15 MPIGlobalToLocal()	44
6.9.1.16 MPILocalToGlobal()	44
6.9.1.17 polyEval()	44
6.9.1.18 polynDerMulti()	45
6.10 /home/renpo/Research/MantleSolver/src/mlweno/util.h File Reference	45
6.10.1 Typedef Documentation	46
6.10.1.1 indice	46
6.10.1.2 indiceSet	46
6.10.1.3 vertex	46
6.10.1.4 vertexSet	46
6.10.2 Function Documentation	46
6.10.2.1 AssignValuesMeshInfo()	47
6.10.2.2 basePoly()	47
6.10.2.3 Bend() [1/2]	47
6.10.2.4 Bend() [2/2]	47
6.10.2.5 constFunc() [1/4]	47
6.10.2.6 constFunc() [2/4]	47
6.10.2.7 constFunc() [3/4]	48
6.10.2.8 constFunc() [4/4]	48
6.10.2.9 delete_pointed_to()	48
6.10.2.10 factorial() [1/2]	48
6.10.2.11 factorial() [2/2]	48
6.10.2.12 FlatIndic() [1/4]	48
6.10.2.13 FlatIndic() [2/4]	49
6.10.2.14 FlatIndic() [3/4]	49
6.10.2.15 FlatIndic() [4/4]	49
6.10.2.16 MPIGlobalToLocal()	49
6.10.2.17 MPILocalToGlobal()	49
6.10.2.18 polyEval()	49
6.10.2.19 polynDerMulti()	50
6.11 util.h	50
Index	53

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

MLWENO	
Weno reconstruction with a given stencil level	7
tensorProductPoly	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

tensorProductPoly::basePolynomial	9
MeshInfo	12
MLWENO::multiLevelReconstruction	16
MLWENO::singleLevelReconstruction	21
stencil< T >	24
tensorProductPoly::stencilPolynomial	29

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

/home/renpo/Research/MantleSolver/src/mlweno/ polynomial.cpp	35
/home/renpo/Research/MantleSolver/src/mlweno/ polynomial.h	35
/home/renpo/Research/MantleSolver/src/mlweno/ reconstruction.cpp	37
/home/renpo/Research/MantleSolver/src/mlweno/ reconstruction.h	37
/home/renpo/Research/MantleSolver/src/mlweno/ stencil.h	39
/home/renpo/Research/MantleSolver/src/mlweno/ util.cpp	41
/home/renpo/Research/MantleSolver/src/mlweno/ util.h	45

Chapter 4

Namespace Documentation

4.1 MLWENO Namespace Reference

Weno reconstruction with a given stencil level.

Classes

- class [multiLevelReconstruction](#)
- class [singleLevelReconstruction](#)

4.1.1 Detailed Description

Weno reconstruction with a given stencil level.

4.2 tensorProductPoly Namespace Reference

Classes

- class [basePolynomial](#)
- class [stencilPolynomial](#)

Chapter 5

Class Documentation

5.1 tensorProductPoly::basePolynomial Class Reference

```
#include <polynomial.h>
```

Public Member Functions

- [basePolynomial](#) ()
- [basePolynomial](#) (const int maxDegree[2])
- [basePolynomial](#) (const int maxDegree[2], double *coef)
- [~basePolynomial](#) ()
- void [setMaxDegree](#) (const int maxDegree[2])
- void [setCoef](#) (double *coef)
- double * [getCoef](#) () const
- int [getSize](#) () const
- int [getI](#) () const
- int [getJ](#) () const
- double [eval](#) (double x, double y) const
- double [eval](#) ([vertex](#) P) const
- double [operator\(\)](#) (double x, double y) const
- double [operator\(\)](#) ([vertex](#) P) const
- double [der](#) (int derX, int derY, double x, double y) const
- double [der](#) (int derX, int derY, [vertex](#) P) const
- void [printCoef](#) () const
- double [getCoef](#) (int i) const

5.1.1 Constructor & Destructor Documentation

5.1.1.1 basePolynomial() [1/3]

```
tensorProductPoly::basePolynomial::basePolynomial ( ) [inline]
```

5.1.1.2 basePolynomial() [2/3]

```
basePolynomial::basePolynomial (
    const int maxDegree[2] )
```

5.1.1.3 basePolynomial() [3/3]

```
basePolynomial::basePolynomial (
    const int maxDegree[2],
    double * coef )
```

5.1.1.4 ~basePolynomial()

```
basePolynomial::~~basePolynomial ( )
```

5.1.2 Member Function Documentation

5.1.2.1 der() [1/2]

```
double tensorProductPoly::basePolynomial::der (
    int derX,
    int derY,
    double x,
    double y ) const
```

5.1.2.2 der() [2/2]

```
double tensorProductPoly::basePolynomial::der (
    int derX,
    int derY,
    vertex P ) const [inline]
```

5.1.2.3 eval() [1/2]

```
double basePolynomial::eval (
    double x,
    double y ) const
```

5.1.2.4 eval() [2/2]

```
double tensorProductPoly::basePolynomial::eval (
    vertex P ) const [inline]
```

5.1.2.5 getCoef() [1/2]

```
double * basePolynomial::getCoef ( ) const
```

5.1.2.6 getCoef() [2/2]

```
double basePolynomial::getCoef (
    int i ) const
```

5.1.2.7 getI()

```
int tensorProductPoly::basePolynomial::getI ( ) const [inline]
```

5.1.2.8 getJ()

```
int tensorProductPoly::basePolynomial::getJ ( ) const [inline]
```

5.1.2.9 getSize()

```
int tensorProductPoly::basePolynomial::getSize ( ) const [inline]
```

5.1.2.10 operator()() [1/2]

```
double tensorProductPoly::basePolynomial::operator() (
    double x,
    double y ) const [inline]
```

5.1.2.11 operator() [2/2]

```
double tensorProductPoly::basePolynomial::operator() (
    vertex P ) const [inline]
```

5.1.2.12 printCoef()

```
void basePolynomial::printCoef ( ) const
```

5.1.2.13 setCoef()

```
void basePolynomial::setCoef (
    double * coef )
```

5.1.2.14 setMaxDegree()

```
void basePolynomial::setMaxDegree (
    const int maxDegree[2] )
```

The documentation for this class was generated from the following files:

- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h](#)
- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp](#)

5.2 MeshInfo Struct Reference

```
#include <util.h>
```

Public Attributes

- int [dim](#)
- vector< int > [MPIlocalCellSize](#)
Total number of dimensions.
- vector< int > [MPIlocalVertexSize](#)
Local chunk size of cell without ghost layer.
- [indice MPIlocalCellStart](#)
Local chunk size of vertex without ghost layer.
- [indice MPIlocalVertexStart](#)
The starting cell index for MPI local part.
- vector< int > [MPIglobalCellSize](#)
The starting vertex index for MPI local part.
- vector< int > [MPIglobalVertexSize](#)
global chunk size of cell without ghost layer
- int [cellGhostLayerSize](#)
global chunk size of vertex without ghost layer
- int [vertexGhostLayerSize](#)
size of ghost layer of cell
- vector< int > [MPIlocalCellSizeFull](#)
size of ghost layer of node
- vector< int > [MPIlocalVertexSizeFull](#)
Local chunk size of cells including ghost layer.
- int [MPIglobalHoriEdgeSize](#)
Local chunk size of vertex including ghost layer.
- int [MPIlocalHoriEdgeSize](#)
- int [MPIglobalVertEdgeSize](#)
- int [MPIlocalVertEdgeSize](#)
- [vertexSet lmesh](#)
- double ** [localVals](#)
- const [indiceSet edgeCorner](#) {{0}, {1}}
- const [indiceSet faceCorner](#) {{0,0},{1,0},{1,1},{0,1}}
- const [indiceSet volumeCorner](#)

5.2.1 Member Data Documentation

5.2.1.1 cellGhostLayerSize

```
int MeshInfo::cellGhostLayerSize
```

global chunk size of vertex without ghost layer

5.2.1.2 dim

```
int MeshInfo::dim
```

In the case, Cell and Vertex are maintained by same global size They will still be stored separately for clarification.

5.2.1.3 edgeCorner

```
const indiceSet MeshInfo::edgeCorner {{0}, {1}}
```

5.2.1.4 faceCorner

```
const indiceSet MeshInfo::faceCorner {{0,0},{1,0},{1,1},{0,1}}
```

5.2.1.5 lmesh

```
vertexSet MeshInfo::lmesh
```

5.2.1.6 localVals

```
double** MeshInfo::localVals
```

5.2.1.7 MPIglobalCellSize

```
vector<int> MeshInfo::MPIglobalCellSize
```

The starting vertex index for MPI local part.

5.2.1.8 MPIglobalHoriEdgeSize

```
int MeshInfo::MPIglobalHoriEdgeSize
```

Local chunk size of vertex including ghost layer.

5.2.1.9 MPIglobalVertEdgeSize

```
int MeshInfo::MPIglobalVertEdgeSize
```

5.2.1.10 MPIglobalVertexSize

```
vector<int> MeshInfo::MPIglobalVertexSize
```

global chunk size of cell without ghost layer

5.2.1.11 MPIlocalCellSize

```
vector<int> MeshInfo::MPIlocalCellSize
```

Total number of dimensions.

5.2.1.12 MPIlocalCellSizeFull

```
vector<int> MeshInfo::MPIlocalCellSizeFull
```

size of ghost layer of node

5.2.1.13 MPIlocalCellStart

```
indice MeshInfo::MPIlocalCellStart
```

Local chunk size of vertex without ghost layer.

5.2.1.14 MPIlocalHoriEdgeSize

```
int MeshInfo::MPIlocalHoriEdgeSize
```

5.2.1.15 MPIlocalVertEdgeSize

```
int MeshInfo::MPIlocalVertEdgeSize
```

5.2.1.16 MPIlocalVertexSize

```
vector<int> MeshInfo::MPIlocalVertexSize
```

Local chunk size of cell without ghost layer.

5.2.1.17 MPIlocalVertexSizeFull

```
vector<int> MeshInfo::MPIlocalVertexSizeFull
```

Local chunk size of cells including ghost layer.

5.2.1.18 MPIlocalVertexStart

```
indice MeshInfo::MPIlocalVertexStart
```

The starting cell index for MPI local part.

5.2.1.19 vertexGhostLayerSize

```
int MeshInfo::vertexGhostLayerSize
```

size of ghost layer of cell

5.2.1.20 volumeCorner

```
const indiceSet MeshInfo::volumeCorner
```

Initial value:

```
{{0,0,0},{1,0,0},{1,1,0},{0,1,0},
                                {0,0,1},{1,0,1},{1,1,1},{0,1,1}}
```

The documentation for this struct was generated from the following file:

- </home/renpo/Research/MantleSolver/src/mlweno/util.h>

5.3 MLWENO::multiLevelReconstruction Class Reference

```
#include <reconstruction.h>
```


Public Member Functions

- [multiLevelReconstruction](#) (const [MeshInfo](#) &mi, int stencilSizeX, int stencilSizeY, vector< [indice](#) > brm)
A constructor.
- [multiLevelReconstruction](#) (const [MeshInfo](#) &mi, int *stencilSize, vector< [indice](#) > brm)
- [multiLevelReconstruction](#) (const [MeshInfo](#) &mi, vector< int * > stencilSizes, vector< vector< [indice](#) > > brms)
- [~multiLevelReconstruction](#) ()
A destructor.
- void [AddLevel](#) (const [MeshInfo](#) &mi, int stencilSizeX, int stencilSizeY, vector< [indice](#) > brm)
Add a single weno reconstruction level to the computation.
- void [AddLevel](#) (const [MeshInfo](#) &mi, int *stencilSize, vector< [indice](#) > brm)
- void [AddLevel](#) (const [MeshInfo](#) &mi, vector< int * > stencilSizes, vector< vector< [indice](#) > > brms)
- void [SeparateBoundaryLayer](#) (const [MeshInfo](#) &mi, const int &layerSize)
Specify boundary layers (cells near boundary that need additional reconstrucion level then interior cells)
- void [SeparateBoundaryLayer](#) (const [MeshInfo](#) &mi)
- void [UpdateNonLinearWgts](#) (const [MeshInfo](#) &mi, const int stage)
- void [ModifyReconstMethod](#) (std::string key, vector< [indice](#) > newRecosntMethod)
- void [ModifyReconstMethod](#) (int stencilSizeX, int stencilSizeY, vector< [indice](#) > newReconstMethod)
- void [SelectWenoReconstLevel](#) (unordered_set< std::string > keys)
- void [GetInfo](#) ()
Routines used to check results and verification.
- void [PrintBoundaryLayer](#) (const [MeshInfo](#) &mi)
- void [PrintSmoothnessIndicator](#) (const [MeshInfo](#) &mi)
- void [PrintNonLinearWgts](#) (const [MeshInfo](#) &mi)
- void [Clear](#) ()

5.3.1 Constructor & Destructor Documentation

5.3.1.1 multiLevelReconstruction() [1/3]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction (
    const MeshInfo & mi,
    int stencilSizeX,
    int stencilSizeY,
    vector< indice > brm ) [inline]
```

A constructor.

! Construt multi-level weno reconstruction by specifying each single level

5.3.1.2 multiLevelReconstruction() [2/3]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction (
    const MeshInfo & mi,
    int * stencilSize,
    vector< indice > brm ) [inline]
```

5.3.1.3 multiLevelReconstruction() [3/3]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction (
    const MeshInfo & mi,
    vector< int * > stencilSizes,
    vector< vector< indice > > brms ) [inline]
```

5.3.1.4 ~multiLevelReconstruction()

```
MLWENO::multiLevelReconstruction::~~multiLevelReconstruction ( ) [inline]
```

A destructor.

Construct multi-level weno reconstruction by specifying each single level

5.3.2 Member Function Documentation

5.3.2.1 AddLevel() [1/3]

```
void MLWENO::multiLevelReconstruction::AddLevel (
    const MeshInfo & mi,
    int * stencilSize,
    vector< indice > brm ) [inline]
```

5.3.2.2 AddLevel() [2/3]

```
void multiLevelReconstruction::AddLevel (
    const MeshInfo & mi,
    int stencilSizeX,
    int stencilSizeY,
    vector< indice > brm )
```

Add a single weno reconstruction level to the computation.

Add a single weno reconstruction level to the weno reconstruction. Create a reference key name for specific level. Create map from the reference key to the single level reconstruction. Create single level key

Check if the new level has never been defined before

Insert new key into allLevels;

Initialize single level reconstruction class pointer.

Create map from single level key to created single level reconstruction

Create map from single level key to reconstruction method (how to find stencils)

Update lowest and highest order of reconstruction level

Accumulate total levels

5.3.2.3 AddLevel() [3/3]

```
void MLWENO::multiLevelReconstruction::AddLevel (
    const MeshInfo & mi,
    vector< int * > stencilSizes,
    vector< vector< indice > > brms ) [inline]
```

5.3.2.4 Clear()

```
void MLWENO::multiLevelReconstruction::Clear ( )
```

5.3.2.5 GetInfo()

```
void multiLevelReconstruction::GetInfo ( )
```

Routines used to check results and verification.

Print added levels and reconstruction methods

5.3.2.6 ModifyReconstMethod() [1/2]

```
void MLWENO::multiLevelReconstruction::ModifyReconstMethod (
    int stencilSizeX,
    int stencilSizeY,
    vector< indice > newReconstMethod ) [inline]
```

5.3.2.7 ModifyReconstMethod() [2/2]

```
void multiLevelReconstruction::ModifyReconstMethod (
    std::string key,
    vector< indice > newReconstMethod )
```

Update reconstruction methods. Called when same level are used, but different reconstruction methods. For example, advection and diffusion reconstruction uses third order level but with different stencils. When an existing level is added, update reconstruction method with new one.

Modify reconst method with given key. Only modify existing reconst method. Please use AddLevel to add reconstruction levels. Make sure it is a existing key

Erasing the existing key

Assign key and reconstruction method pair

5.3.2.8 PrintBoundaryLayer()

```
void multiLevelReconstruction::PrintBoundaryLayer (
    const MeshInfo & mi )
```

5.3.2.9 PrintNonLinearWgts()

```
void multiLevelReconstruction::PrintNonLinearWgts (
    const MeshInfo & mi )
```

5.3.2.10 PrintSmoothnessIndicator()

```
void multiLevelReconstruction::PrintSmoothnessIndicator (
    const MeshInfo & mi )
```

5.3.2.11 SelectWenoReconstLevel()

```
void multiLevelReconstruction::SelectWenoReconstLevel (
    unordered_set< std::string > keys )
```

Select Weno reconstruction levels to be used in the reconstruction.

Modify wenoLevels_. Selecting predefined reconstruction method in the actual computation process. Make wenoLevels_ an empty set

Insert selected keys

5.3.2.12 SeparateBoundaryLayer() [1/2]

```
void multiLevelReconstruction::SeparateBoundaryLayer (
    const MeshInfo & mi )
```

5.3.2.13 SeparateBoundaryLayer() [2/2]

```
void multiLevelReconstruction::SeparateBoundaryLayer (
    const MeshInfo & mi,
    const int & layerSize )
```

Specify boundary layers (cells near boundary that need additional reconstrucion level then interior cells)

Specify boundary layers (cells near boundary that need additional reconstrucion level then interior cells) Default boundary layer size is set to be 1. Call the second function when a boundary layer size larger than 1 is needed. Create two different weno reconstruction levels for interior and boundary cells.

5.3.2.14 UpdateNonLinearWgts()

```
void multiLevelReconstruction::UpdateNonLinearWgts (
    const MeshInfo & mi,
    const int stage )
```

Update non linear weights for all levels. Smoothness indicator will be updated every time when non linear weights updated.

The documentation for this class was generated from the following files:

- /home/renpo/Research/MantleSolver/src/mlweno/[reconstruction.h](#)
- /home/renpo/Research/MantleSolver/src/mlweno/[reconstruction.cpp](#)

5.4 MLWENO::singleLevelReconstruction Class Reference

```
#include <reconstruction.h>
```

Public Member Functions

- [singleLevelReconstruction](#) ()
A constructor.
- [singleLevelReconstruction](#) (int stencilSizeX, int stencilSizeY)
A constructor.
- [~singleLevelReconstruction](#) ()
A destructor.
- void [CreateStencilPolynomials](#) (const [MeshInfo](#) &mi)
- bool [CheckExist](#) (const [MeshInfo](#) &mi, [indice](#) owner) const
- const int [GetSizeX](#) () const
- const int [GetSizeY](#) () const
- const double [GetScale](#) (int s)
- const double [GetScale](#) (const [MeshInfo](#) &mi, [indice](#) owner)
- double [CalculateSmoothnessIndic](#) (const [MeshInfo](#) &mi, [indice](#) owner)
Calculate smoothness indicator for the single stencil.
- double [GetSmoothnessIndic](#) (const [MeshInfo](#) &mi, [indice](#) owner)
Extract smoothness indicator from pre-calculated values.
- void [UpdateSmoothnessIndic](#) (const [MeshInfo](#) &mi)
Update smoothness indicator for entire reconstruction level.
- void [CheckStencils](#) () const
class members for checking and verification
- void [CheckStencilPolynomials](#) (const [MeshInfo](#) &mi, [indice](#) start)
Functions checking created stencil polynomials for a single stencil.
- void [PrintSmoothnessIndicator](#) (const [MeshInfo](#) &mi)
Print all smoothnessIndicator for all stencils belonging to this level.

5.4.1 Constructor & Destructor Documentation

5.4.1.1 singleLevelReconstruction() [1/2]

```
MLWENO::singleLevelReconstruction::singleLevelReconstruction ( ) [inline]
```

A constructor.

Default constructor for a single reconstruction level.

5.4.1.2 singleLevelReconstruction() [2/2]

```
singleLevelReconstruction::singleLevelReconstruction (
    int stencilSizeX,
    int stencilSizeY )
```

A constructor.

Costume constructor with given x and y stencil size. Create stencil of indice used in creating stencil polynomials

5.4.1.3 ~singleLevelReconstruction()

```
MLWENO::singleLevelReconstruction::~~singleLevelReconstruction ( ) [inline]
```

A destructor.

Clear interior, single level polynomial and smoothness indicators

5.4.2 Member Function Documentation

5.4.2.1 CalculateSmoothnessIndic()

```
double singleLevelReconstruction::CalculateSmoothnessIndic (
    const MeshInfo & mi,
    indice owner )
```

Calculate smoothness indicator for the single stencil.

Directly calculate smoothness indicator of a given stencil Should not be called directly for computational efficiency

5.4.2.2 CheckExist()

```
bool MLWENO::singleLevelReconstruction::CheckExist (
    const MeshInfo & mi,
    indice owner ) const [inline]
```

5.4.2.3 CheckStencilPolynomials()

```
void singleLevelReconstruction::CheckStencilPolynomials (
    const MeshInfo & mi,
    indice start )
```

Functions checking created stencil polynomials for a single stencil.

5.4.2.4 CheckStencils()

```
void MLWENO::singleLevelReconstruction::CheckStencils ( ) const [inline]
```

class members for checking and verification

5.4.2.5 CreateStencilPolynomials()

```
void singleLevelReconstruction::CreateStencilPolynomials (
    const MeshInfo & mi )
```

! Two steps of creating stencil polynomials for a given reconstruction level. Step 1: Identify all the stencils covering entire computational domain. Step 2: Compute stencil polynomials for each stencil.

5.4.2.6 GetScale() [1/2]

```
const double MLWENO::singleLevelReconstruction::GetScale (
    const MeshInfo & mi,
    indice owner ) [inline]
```

5.4.2.7 GetScale() [2/2]

```
const double MLWENO::singleLevelReconstruction::GetScale (
    int s ) [inline]
```

5.4.2.8 GetSizeX()

```
const int MLWENO::singleLevelReconstruction::GetSizeX ( ) const [inline]
```

5.4.2.9 GetSizeY()

```
const int MLWENO::singleLevelReconstruction::GetSizeY ( ) const [inline]
```

5.4.2.10 GetSmoothnessIndic()

```
double singleLevelReconstruction::GetSmoothnessIndic (
    const MeshInfo & mi,
    indice owner )
```

Extract smoothness indicator from pre-calculated values.

Directly extract pre-calculated smoothness indicator. Should always be the one to call when smoothness indicator is needed.

5.4.2.11 PrintSmoothnessIndicator()

```
void singleLevelReconstruction::PrintSmoothnessIndicator (
    const MeshInfo & mi )
```

Print all smoothnessIndicator for all stencils belonging to this level.

5.4.2.12 UpdateSmoothnessIndic()

```
void singleLevelReconstruction::UpdateSmoothnessIndic (
    const MeshInfo & mi )
```

Update smoothness indicator for entire reconstruction level.

Update smoothness indicators for the entire level.

The documentation for this class was generated from the following files:

- [/home/renpo/Research/MantleSolver/src/mlweno/reconstruction.h](#)
- [/home/renpo/Research/MantleSolver/src/mlweno/reconstruction.cpp](#)

5.5 stencil< T > Class Template Reference

```
#include <stencil.h>
```


Public Member Functions

- [stencil](#) ()
- [stencil](#) (size_t I)
- [stencil](#) (size_t I, size_t J)
- [stencil](#) (size_t I, size_t J, size_t K)
- [~stencil](#) ()
- void [SetStencil](#) (size_t I)
- void [SetStencil](#) (size_t I, size_t J)
- void [SetStencil](#) (size_t I, size_t J, size_t K)
- void [CreateStencil](#) ()
- const int [getSize](#) () const
- const int [getI](#) () const
- const int [getJ](#) () const
- const int [getK](#) () const
- void [setI](#) (size_t I) const
- void [setJ](#) (size_t J) const
- void [setK](#) (size_t K) const
- T & [operator\(\)](#) (size_t i)
- T & [operator\(\)](#) (size_t i, size_t j)
- T & [operator\(\)](#) (size_t i, size_t j, size_t k)
- T [operator\(\)](#) (size_t i) const
- T [operator\(\)](#) (size_t i, size_t j) const
- T [operator\(\)](#) (size_t i, size_t j, size_t k) const
- const vector< T > & [GetStencil](#) () const
- void [clearPtr](#) ()

5.5.1 Constructor & Destructor Documentation

5.5.1.1 stencil() [1/4]

```
template<class T >
stencil< T >::stencil ( ) [inline]
```

5.5.1.2 stencil() [2/4]

```
template<class T >
stencil< T >::stencil (
    size_t I ) [inline]
```

5.5.1.3 stencil() [3/4]

```
template<class T >
stencil< T >::stencil (
    size_t I,
    size_t J ) [inline]
```

5.5.1.4 stencil() [4/4]

```
template<class T >
stencil< T >::stencil (
    size_t I,
    size_t J,
    size_t K ) [inline]
```

5.5.1.5 ~stencil()

```
template<class T >
stencil< T >::~stencil ( ) [inline]
```

5.5.2 Member Function Documentation

5.5.2.1 clearPtr()

```
template<class T >
void stencil< T >::clearPtr ( ) [inline]
```

5.5.2.2 CreateStencil()

```
template<class T >
void stencil< T >::CreateStencil ( ) [inline]
```

5.5.2.3 getI()

```
template<class T >
const int stencil< T >::getI ( ) const [inline]
```

5.5.2.4 getJ()

```
template<class T >
const int stencil< T >::getJ ( ) const [inline]
```

5.5.2.5 getK()

```
template<class T >
const int stencil< T >::getK ( ) const [inline]
```

5.5.2.6 getSize()

```
template<class T >
const int stencil< T >::getSize ( ) const [inline]
```

5.5.2.7 GetStencil()

```
template<class T >
const vector< T > & stencil< T >::GetStencil ( ) const [inline]
```

5.5.2.8 operator>() [1/6]

```
template<class T >
T & stencil< T >::operator() (
    size_t i ) [inline]
```

5.5.2.9 operator>() [2/6]

```
template<class T >
T stencil< T >::operator() (
    size_t i ) const [inline]
```

5.5.2.10 operator>() [3/6]

```
template<class T >
T & stencil< T >::operator() (
    size_t i,
    size_t j ) [inline]
```

5.5.2.11 operator>() [4/6]

```
template<class T >
T stencil< T >::operator() (
    size_t i,
    size_t j ) const [inline]
```

5.5.2.12 operator>() [5/6]

```
template<class T >
T & stencil< T >::operator() (
    size_t i,
    size_t j,
    size_t k ) [inline]
```

5.5.2.13 operator>() [6/6]

```
template<class T >
T stencil< T >::operator() (
    size_t i,
    size_t j,
    size_t k ) const [inline]
```

5.5.2.14 setI()

```
template<class T >
void stencil< T >::setI (
    size_t I ) const [inline]
```

5.5.2.15 setJ()

```
template<class T >
void stencil< T >::setJ (
    size_t J ) const [inline]
```

5.5.2.16 setK()

```
template<class T >
void stencil< T >::setK (
    size_t K ) const [inline]
```

5.5.2.17 SetStencil() [1/3]

```
template<class T >
void stencil< T >::SetStencil (
    size_t I ) [inline]
```

5.5.2.18 SetStencil() [2/3]

```
template<class T >
void stencil< T >::SetStencil (
    size_t I,
    size_t J ) [inline]
```

5.5.2.19 SetStencil() [3/3]

```
template<class T >
void stencil< T >::SetStencil (
    size_t I,
    size_t J,
    size_t K ) [inline]
```

The documentation for this class was generated from the following file:

- </home/renpo/Research/MantleSolver/src/mlweno/stencil.h>

5.6 tensorProductPoly::stencilPolynomial Class Reference

```
#include <polynomial.h>
```

Public Member Functions

- [stencilPolynomial](#) ()
- [stencilPolynomial](#) (const [indice](#) &start, const [vertex](#) ¢er)
- [stencilPolynomial](#) (const [indice](#) &start, const [vertex](#) ¢er, const vector< [indice](#) > &targetCell)
- [~stencilPolynomial](#) ()
- void [SetStencilPolynomials](#) (const [MeshInfo](#) &mi, const [stencil](#)< [indice](#) > &stencilIndice)
- void [SetUpScale](#) (const [MeshInfo](#) &mi, const vector< [indice](#) > &targetCell)
- void [SetUpScale](#) (const [MeshInfo](#) &mi, const [stencil](#)< [indice](#) > &stencilIndice)
- const double [GetScale](#) () const
- double [eval](#) (const double x, const double y) const
- double [eval](#) (const [vertex](#) &P) const
- double [operator\(\)](#) (const double x, const double y) const
- double [operator\(\)](#) (const [vertex](#) &P) const
- void [printCoef](#) ()
- void [printCoef](#) (int s)
- void [SetCollapsePolyn](#) (const [MeshInfo](#) &mi, const [stencil](#)< [indice](#) > &stencilIndice)
- double [GetSmoothIndic](#) (const [MeshInfo](#) &mi, const [stencil](#)< [indice](#) > &stencilIndice)
- const int [GetOrderX](#) () const
- const int [GetOrderY](#) () const

5.6.1 Constructor & Destructor Documentation

5.6.1.1 stencilPolynomial() [1/3]

```
tensorProductPoly::stencilPolynomial::stencilPolynomial ( ) [inline]
```

5.6.1.2 stencilPolynomial() [2/3]

```
stencilPolynomial::stencilPolynomial (
    const indice & start,
    const vertex & center )
```

5.6.1.3 stencilPolynomial() [3/3]

```
stencilPolynomial::stencilPolynomial (
    const indice & start,
    const vertex & center,
    const vector< indice > & targetCell )
```

5.6.1.4 ~stencilPolynomial()

```
tensorProductPoly::stencilPolynomial::~~stencilPolynomial ( ) [inline]
```

5.6.2 Member Function Documentation

5.6.2.1 eval() [1/2]

```
double stencilPolynomial::eval (
    const double x,
    const double y ) const
```

5.6.2.2 eval() [2/2]

```
double tensorProductPoly::stencilPolynomial::eval (
    const vertex & P ) const [inline]
```

5.6.2.3 GetOrderX()

```
const int tensorProductPoly::stencilPolynomial::GetOrderX ( ) const [inline]
```

5.6.2.4 GetOrderY()

```
const int tensorProductPoly::stencilPolynomial::GetOrderY ( ) const [inline]
```

5.6.2.5 GetScale()

```
const double tensorProductPoly::stencilPolynomial::GetScale ( ) const [inline]
```

5.6.2.6 GetSmoothIndic()

```
double stencilPolynomial::GetSmoothIndic (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndices )
```

5.6.2.7 operator() [1/2]

```
double tensorProductPoly::stencilPolynomial::operator() (
    const double x,
    const double y ) const [inline]
```

5.6.2.8 operator() [2/2]

```
double tensorProductPoly::stencilPolynomial::operator() (
    const vertex & P ) const [inline]
```

5.6.2.9 printCoef() [1/2]

```
void stencilPolynomial::printCoef ( )
```

5.6.2.10 printCoef() [2/2]

```
void stencilPolynomial::printCoef (
    int s )
```

5.6.2.11 SetCollapsePolyn()

```
void stencilPolynomial::SetCollapsePolyn (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndice )
```

5.6.2.12 SetStencilPolynomials()

```
void stencilPolynomial::SetStencilPolynomials (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndice )
```


5.6.2.13 SetUpScale() [1/2]

```
void tensorProductPoly::stencilPolynomial::SetUpScale (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndice ) [inline]
```

5.6.2.14 SetUpScale() [2/2]

```
void tensorProductPoly::stencilPolynomial::SetUpScale (
    const MeshInfo & mi,
    const vector< indice > & targetCell ) [inline]
```

The documentation for this class was generated from the following files:

- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h](#)
- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp](#)

Chapter 6

File Documentation

6.1 `/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp` File Reference

```
#include "polynomial.h"
```

6.2 `/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h` File Reference

```
#include "util.h"  
#include "stencil.h"
```

Classes

- class [tensorProductPoly::basePolynomial](#)
- class [tensorProductPoly::stencilPolynomial](#)

Namespaces

- namespace [tensorProductPoly](#)

6.3 polynomial.h

[Go to the documentation of this file.](#)

```

00001 #ifndef POLYNOMIAL_H_
00002 #define POLYNOMIAL_H_
00003
00004 #include "util.h"
00005 #include "stencil.h"
00006
00007 namespace tensorProductPoly{
00008     // Tensor product 2D polynomial for stencil basis
00009     class basePolynomial {
00010     public:
00011         basePolynomial() {};
00012         basePolynomial(const int maxDegree[2]);
00013         basePolynomial(const int maxDegree[2], double* coef);
00014         ~basePolynomial();
00015
00016         void setMaxDegree(const int maxDegree[2]);
00017         void setCoef(double* coef);
00018
00019         double* getCoef() const;
00020
00021         // Return the size of the polynomial
00022         int getSize() const {return maxDegree[0]*maxDegree[1];};
00023
00024         // Return the size of degree x as I
00025         int getI() const {return maxDegree[0];};
00026         // Return the size of degree y as J
00027         int getJ() const {return maxDegree[1];};
00028
00029         // Evaluation of point value for a given polynomial
00030         double eval(double x, double y) const;
00031         double eval(vertex P) const {return eval(P[0],P[1]);};
00032         double operator() (double x, double y) const {return eval(x,y);};
00033         double operator() (vertex P) const {return eval(P);};
00034
00035         // Evaluation of derivative value for a given polynomial
00036         double der(int derX, int derY, double x, double y) const;
00037         double der(int derX, int derY, vertex P) const {return der(derX, derY, P[0], P[1]);};
00038
00039         // Print coefficients out
00040         void printCoef() const;
00041
00042         double getCoef(int i) const;
00043
00044     private:
00045         int maxDegree_[2] = {-1,-1};
00046         double* coef_ = nullptr;
00047     };
00048
00049 // stencil polynomials used for weno reconstruction
00050 class stencilPolynomial {
00051 public:
00052     stencilPolynomial() {};
00053     stencilPolynomial(const indice& start, const vertex& center);
00054     stencilPolynomial(const indice& start, const vertex& center,
00055                     const vector<indice>& targetCell);
00056     ~stencilPolynomial() {stencilPolyn_.clearPtr();
00057                         delete [] collapsePolyn_};
00058
00059     void SetStencilPolynomials(const MeshInfo& mi,
00060                             const stencil <indice>& stencilIndice);
00061
00062     void SetUpScale(const MeshInfo& mi, const vector<indice>& targetCell)
00063         {SetTargetCell_(targetCell);ComputeCellBasedScale_(mi);};
00064     void SetUpScale(const MeshInfo& mi, const stencil <indice>& stencilIndice)
00065         {ComputeStencilBasedScale_(mi, stencilIndice);};
00066
00067     const double GetScale() const {return scale_};
00068
00069     double eval(const double x, const double y) const;
00070     double eval(const vertex& P) const {return eval(P[0],P[1]);};
00071     double operator() (const double x, const double y) const {return eval(x,y);};
00072     double operator() (const vertex& P) const {return eval(P);};
00073
00074     // Check basis polynomial coefficients
00075     void printCoef();
00076     void printCoef(int s);
00077
00078     // Set collapse polynomial
00079     void SetCollapsePolyn(const MeshInfo& mi, const stencil <indice>& stencilIndice);

```

```

00082
00083     double GetSmoothIndic(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00084
00085     const int GetOrderX() const { return stencilPolyn_.getI(); };
00086     const int GetOrderY() const { return stencilPolyn_.getJ(); };
00087
00088     private:
00089
00090     vertex center_ = {0.0,0.0};
00091     indice start_ = {-1,-1};
00092
00093     void SetTargetCell_(const vector<indice>& targetCell);
00094     vector<indice> targetCell_;
00095     double scale_ = -1.0;
00096
00097     double coef_ = 1.0;
00098
00099     // Calculate scale based on the information of a single cell
00100     void ComputeCellBasedScale_(const MeshInfo& mi);
00101     // Calculate scale based on the value of a given stencil
00102     void ComputeStencilBasedScale_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00103
00104     stencil <basePolynomial*> stencilPolyn_;
00105
00106     void SetCollapsePolyn_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00107     basePolynomial* collapsePolyn_ = nullptr;
00108
00109     // Create polynomial smoothness indicator
00110     void CreateXi_();
00111     vector<double> Xi_;
00112
00113     int maxR_;
00114     int minR_;
00115
00116     void EvalSmoothIndic_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00117     double smoothnessIndic_ = -1;
00118
00119 };
00120
00121 // End of using name space MLWENO
00122 }
00123
00124 #endif

```

6.4 /home/renpo/Research/MantleSolver/src/mlweno/reconstruction.cpp File Reference

```
#include "reconstruction.h"
```

6.5 /home/renpo/Research/MantleSolver/src/mlweno/reconstruction.h File Reference

```
#include "polynomial.h"
#include <map>
```

Classes

- class [MLWENO::singleLevelReconstruction](#)
- class [MLWENO::multiLevelReconstruction](#)

Namespaces

- namespace [MLWENO](#)

Weno reconstruction with a given stencil level.

6.6 reconstruction.h

[Go to the documentation of this file.](#)

```

00001 #ifndef RECONSTRUCTION_H_
00002 #define RECONSTRUCTION_H_
00003
00004 //\slreconstruction.h slreconstruction.h
00005
00006 #include "polynomial.h"
00007 #include <map>
00008
00009 namespace MLWENO{
00010
00011     class singleLevelReconstruction {
00012     public:
00013
00014         singleLevelReconstruction() {};;
00015
00016         singleLevelReconstruction(int stencilSizeX, int stencilSizeY);
00017
00018         ~singleLevelReconstruction() {interior_.clear();
00019 singleLevel_.clear();smoothnessIndic_.clear();};
00020
00021         // =====
00022         void CreateStencilPolynomials(const MeshInfo& mi);
00023
00024         bool CheckExist(const MeshInfo& mi, indice owner) const {return
00025 interior_.count(FlatIndic(mi,owner));};
00026
00027         const int GetSizeX() const {return stencilSizeX_};;
00028         const int GetSizeY() const {return stencilSizeY_};;
00029
00030         const double GetScale(int s) {return singleLevel_[s]->GetScale();};
00031         const double GetScale(const MeshInfo& mi, indice owner) {return
00032 singleLevel_[FlatIndic(mi,owner)]->GetScale();};
00033
00034         double CalculateSmoothnessIndic(const MeshInfo& mi, indice owner);
00035
00036         double GetSmoothnessIndic(const MeshInfo& mi, indice owner);
00037
00038         void UpdateSmoothnessIndic(const MeshInfo& mi);
00039
00040         // =====
00041         void CheckStencils() const {cout<< "Constructed "<< interior_.size() << " stencils with the
00042 size of " << stencilSizeX_ << " " << stencilSizeY_ << endl;};
00043         void CheckStencilPolynomials(const MeshInfo& mi, indice start);
00044         void PrintSmoothnessIndicator(const MeshInfo& mi);
00045
00046     private:
00047
00048         int stencilSizeX_ = -1;
00049         int stencilSizeY_ = -1;
00050
00051         stencil <indice> stencilIndice_;
00052         unordered_set<int> interior_;
00053         map<int, double> smoothnessIndic_;
00054         map<int, tensorProductPoly::stencilPolynomial*> singleLevel_;
00055
00056         void IdentifyInteriorCell_(const MeshInfo& mi);
00057
00058         const vertex ComputeStencilCenter_(const MeshInfo& mi, int flat);
00059
00060         void ComputeStencilPolyn_(const MeshInfo& mi);
00061     };
00062
00063     class multiLevelReconstruction {
00064     public:
00065
00066         //multiLevelReconstruction() {};;
00067
00068         multiLevelReconstruction(const MeshInfo& mi, int stencilSizeX, int stencilSizeY,
00069 vector<indice> brm)
00070 {AddLevel(mi,stencilSizeX,stencilSizeY,brm);};

```

```

00089
00090     multiLevelReconstruction(const MeshInfo& mi, int* stencilSize, vector<indice> brm)
00091     {AddLevel(mi, stencilSize, brm);};
00092
00093     multiLevelReconstruction(const MeshInfo& mi, vector<int*> stencilSizes,
00094                             vector<vector<indice>> brms)
00095     {AddLevel(mi, stencilSizes, brms);};
00096
00097
00098
00099     ~multiLevelReconstruction() {Clear();};
00100
00101
00102     void AddLevel(const MeshInfo& mi, int stencilSizeX, int stencilSizeY, vector<indice> brm);
00103     void AddLevel(const MeshInfo& mi, int* stencilSize, vector<indice> brm)
00104     {AddLevel(mi, stencilSize[0], stencilSize[1], brm);};
00105     void AddLevel(const MeshInfo& mi, vector<int*> stencilSizes, vector<vector<indice>> brms)
00106     {for (int i=0; i<stencilSizes.size(); i++){
00107         AddLevel(mi, stencilSizes[i], brms[i]);};};
00108
00109     void SeparateBoundaryLayer(const MeshInfo& mi, const int& layerSize);
00110     void SeparateBoundaryLayer(const MeshInfo& mi);
00111
00112     void UpdateNonLinearWgts(const MeshInfo& mi, const int stage);
00113
00114     void ModifyReconstMethod(std::string key, vector<indice> newReconstMethod);
00115     void ModifyReconstMethod(int stencilSizeX, int stencilSizeY, vector<indice>
00116                             newReconstMethod)
00117     {std::string key = '(' + std::to_string(stencilSizeX) + ',' + std::to_string(stencilSizeY)
00118     + ')';
00119         ModifyReconstMethod(key, newReconstMethod);};
00120
00121     void SelectWenoReconstLevel(unordered_set<std::string> keys);
00122
00123     void GetInfo();
00124     void PrintBoundaryLayer(const MeshInfo& mi);
00125     void PrintSmoothnessIndicator(const MeshInfo& mi);
00126     void PrintNonLinearWgts(const MeshInfo& mi);
00127
00128     void Clear();
00129 private:
00130     const double eps0_ = 0.01;
00131
00132     std::string lowestLevel_;
00133     std::string highestLevel_;
00134
00135     map<std::string, singleLevelReconstruction*> reconstLevels_;
00136     map<std::string, vector<indice>> reconstMethods_;
00137
00138     unordered_set<int> interiorCells_;
00139     unordered_set<int> boundaryCells_;
00140
00141     unordered_set<std::string> wenoLevels_;
00142     unordered_set<std::string> interiorLevels_;
00143     unordered_set<std::string> boundaryLevels_;
00144
00145     int totalLevels_ = 0;
00146
00147     unordered_map<int, unordered_map<std::string, unordered_map<int, double>> nonLinearWgts_>
00148
00149     void SeparateReconstMethods_();
00150
00151     void UpdateOneStageNonLinearWgts_(const MeshInfo& mi, int flatGlobal,
00152                                     const unordered_set<std::string>& levels);
00153
00154     void UpdateTwoStageNonLinearWgts_(const MeshInfo& mi, int flatGlobal,
00155                                     const unordered_set<std::string>& levels);
00156
00157     vector< map<int, int> > etaBias_;
00158 };
00159
00160 #endif

```

6.7 /home/renpo/Research/MantleSolver/src/mlweno/stencil.h File Reference

```
#include "util.h"
```

Classes

- class [stencil< T >](#)

Enumerations

- enum [stencilType](#) { [cellCentered](#) , [edgeCentered](#) , [vertexCentered](#) }

6.7.1 Enumeration Type Documentation

6.7.1.1 stencilType

enum [stencilType](#)

Enumerator

cellCentered	
edgeCentered	
vertexCentered	

6.8 stencil.h

[Go to the documentation of this file.](#)

```

00001 #ifndef STENCIL_H_
00002 #define STENCIL_H_
00003
00004 /*
00005  *Define multi-level stencils for weno reconstruction
00006  *Designed for cartesian logically rectangular mesh.
00007  */
00008
00009 #include "util.h"
00010
00011
00012 enum stencilType {cellCentered, edgeCentered, vertexCentered};
00013
00014 template <class T>
00015 class stencil{
00016     public:
00017         stencil() {};
00018         stencil(size_t I):                I_(I), stencilSize_(I)
00019         {stencil_.resize(stencilSize_)};
00019         stencil(size_t I, size_t J):      I_(I), J_(J), stencilSize_(I*J)
00020         {stencil_.resize(stencilSize_)};
00020         stencil(size_t I, size_t J, size_t K): I_(I), J_(J), K_(K), stencilSize_(I*J*K)
00021         {stencil_.resize(stencilSize_)};
00021
00022         ~stencil() {};
00023
00024         void SetStencil(size_t I) {I_ = I; stencilSize_ = I; stencil_.resize(stencilSize_);}
00025         void SetStencil(size_t I, size_t J) {I_ = I; J_ = J; stencilSize_ = I*J;
00026         stencil_.resize(stencilSize_);}
00026         void SetStencil(size_t I, size_t J, size_t K) {I_ = I; J_ = J; K_ = K; stencilSize_ = I*J*K;
00027         stencil_.resize(stencilSize_);}
00027
00028         void CreateStencil() {stencil_.resize(stencilSize_);}
00029
00030         const int getSize() const {return stencilSize_};

```



```

00031
00032     const int getI() const {return I_};
00033     const int getJ() const {return J_};
00034     const int getK() const {return K_};
00035
00036     void setI(size_t I) const {I_ = I};
00037     void setJ(size_t J) const {J_ = J};
00038     void setK(size_t K) const {K_ = K};
00039
00040     T &operator()(size_t i){
00041         return stencil_[i];
00042     }
00043
00044     T &operator()(size_t i, size_t j){
00045         return stencil_[i+j*I_];
00046     }
00047
00048     T &operator()(size_t i, size_t j, size_t k){
00049         return stencil_[i+j*I_+k*I_*J_];
00050     }
00051
00052     T operator()(size_t i) const{
00053         return stencil_[i];
00054     }
00055
00056     T operator()(size_t i, size_t j) const{
00057         return stencil_[i+j*I_];
00058     }
00059
00060     T operator()(size_t i, size_t j, size_t k) const{
00061         return stencil_[i+j*I_+k*I_*J_];
00062     }
00063
00064     const vector<T>& GetStencil() const {return stencil_};
00065
00066     // Call it when T is pointer
00067     void clearPtr() {if (std::is_pointer<T>::value){
00068         std::for_each(stencil_.begin(), stencil_.end(), delete_pointed_to<T>);}}
00069
00070     private:
00071         size_t I_;
00072         size_t J_;
00073         size_t K_;
00074         size_t stencilSize_;
00075         vector<T> stencil_;
00076 };
00077
00078 #endif

```

6.9 /home/renpo/Research/MantleSolver/src/mlweno/util.cpp File Reference

```
#include "util.h"
```

Functions

- double [constFunc](#) (valarray< double > &point, const vector< double > ¶m)
- double [constFunc](#) (valarray< double > &point, const vector< double > ¶m, double c)
- double [constFunc](#) ()
- double [constFunc](#) (double c)
- double [basePoly](#) ([vertex](#) &point, const vector< int > ¶m)
- int [factorial](#) (int top, int bottom)
- int [factorial](#) (int top)
- double [polyEval](#) (double x, double *coef, int degree)
- void [polynDerMulti](#) (int der, int max, int *multiplier)
- [indice MPILocalToGlobal](#) ([indice](#) local, const [MeshInfo](#) &mi)
- [indice MPIGlobalToLocal](#) ([indice](#) global, const [MeshInfo](#) &mi)
- int [FlatIndic](#) (const [MeshInfo](#) &mi, int i, int j)

- int [FlatIndic](#) (const int M, int i, int j)
- int [FlatIndic](#) (const [MeshInfo](#) &mi, const [indice](#) &p)
- int [FlatIndic](#) (const int M, const [indice](#) &p)
- [indice Bend](#) (const [MeshInfo](#) &mi, int flat)
- [indice Bend](#) (const int M, int flat)
- void [AssignValuesMeshInfo](#) ([MeshInfo](#) &mi, DM dmv, DM dmu)

6.9.1 Function Documentation

6.9.1.1 AssignValuesMeshInfo()

```
void AssignValuesMeshInfo (
    MeshInfo & mi,
    DM dmv,
    DM dmu )
```

6.9.1.2 basePoly()

```
double basePoly (
    vertex & point,
    const vector< int > & param )
```

6.9.1.3 Bend() [1/2]

```
indice Bend (
    const int M,
    int flat )
```

6.9.1.4 Bend() [2/2]

```
indice Bend (
    const MeshInfo & mi,
    int flat )
```

6.9.1.5 constFunc() [1/4]

```
double constFunc ( )
```

6.9.1.6 constFunc() [2/4]

```
double constFunc (
    double c )
```

6.9.1.7 constFunc() [3/4]

```
double constFunc (
    valarray< double > & point,
    const vector< double > & param )
```

6.9.1.8 constFunc() [4/4]

```
double constFunc (
    valarray< double > & point,
    const vector< double > & param,
    double c )
```

6.9.1.9 factorial() [1/2]

```
int factorial (
    int top )
```

6.9.1.10 factorial() [2/2]

```
int factorial (
    int top,
    int bottom )
```

6.9.1.11 FlatIndic() [1/4]

```
int FlatIndic (
    const int M,
    const indice & p )
```

6.9.1.12 FlatIndic() [2/4]

```
int FlatIndic (
    const int M,
    int i,
    int j )
```

6.9.1.13 FlatIndic() [3/4]

```
int FlatIndic (
    const MeshInfo & mi,
    const indice & p )
```

6.9.1.14 FlatIndic() [4/4]

```
int FlatIndic (
    const MeshInfo & mi,
    int i,
    int j )
```

6.9.1.15 MPIGlobalToLocal()

```
indice MPIGlobalToLocal (
    indice global,
    const MeshInfo & mi )
```

6.9.1.16 MPILocalToGlobal()

```
indice MPILocalToGlobal (
    indice local,
    const MeshInfo & mi )
```

6.9.1.17 polyEval()

```
double polyEval (
    double x,
    double * coef,
    int degree )
```

6.9.1.18 polynDerMulti()

```
void polynDerMulti (
    int der,
    int max,
    int * multiplier )
```

6.10 /home/renpo/Research/MantleSolver/src/mlweno/util.h File Reference

```
#include <vector>
#include <utility>
#include <set>
#include <unordered_set>
#include <array>
#include <valarray>
#include <algorithm>
#include <numeric>
#include <memory>
#include <cstdlib>
#include <type_traits>
#include <iomanip>
#include "lapacke.h"
#include "integral.h"
#include <assert.h>
#include <petsc.h>
```

Classes

- struct [MeshInfo](#)

Typedefs

- using [vertex](#) = valarray< double >
- using [indice](#) = valarray< int >
- using [vertexSet](#) = vector< valarray< double > >
- using [indiceSet](#) = vector< valarray< double > >

Functions

- template<typename T >
void [delete_pointed_to](#) (T const ptr)
- void [AssignValuesMeshInfo](#) ([MeshInfo](#) &mi, DM dmv, DM dmu)
- double [constFunc](#) (valarray< double > &point, const vector< double > ¶m)
- double [constFunc](#) (valarray< double > &point, const vector< double > ¶m, double c)
- double [constFunc](#) ()
- double [constFunc](#) (double c)
- int [factorial](#) (int top, int bottom)
- int [factorial](#) (int top)

- double `basePoly` (`vertex` &point, const vector< int > ¶m)
- double `polyEval` (double x, double *coef, int degree)
- void `polynDerMulti` (int der, int max, int *multiplier)
- `indice MPILocalToGlobal` (`indice` local, const `MeshInfo` &mi)
- `indice MPIGlobalToLocal` (`indice` global, const `MeshInfo` &mi)
- int `FlatIndic` (const `MeshInfo` &mi, int i, int j)
- int `FlatIndic` (const int M, int i, int j)
- int `FlatIndic` (const `MeshInfo` &mi, const `indice` &p)
- int `FlatIndic` (const int M, const `indice` &p)
- `indice Bend` (const `MeshInfo` &mi, int flat)
- `indice Bend` (const int M, int flat)

6.10.1 Typedef Documentation

6.10.1.1 indice

```
using indice = valarray<int>
```

6.10.1.2 indiceSet

```
using indiceSet = vector<valarray<double> >
```

6.10.1.3 vertex

```
using vertex = valarray<double>
```

6.10.1.4 vertexSet

```
using vertexSet = vector<valarray<double> >
```

6.10.2 Function Documentation

6.10.2.1 AssignValuesMeshInfo()

```
void AssignValuesMeshInfo (
    MeshInfo & mi,
    DM dmv,
    DM dmu )
```

6.10.2.2 basePoly()

```
double basePoly (
    vertex & point,
    const vector< int > & param )
```

6.10.2.3 Bend() [1/2]

```
indice Bend (
    const int M,
    int flat )
```

6.10.2.4 Bend() [2/2]

```
indice Bend (
    const MeshInfo & mi,
    int flat )
```

6.10.2.5 constFunc() [1/4]

```
double constFunc ( )
```

6.10.2.6 constFunc() [2/4]

```
double constFunc (
    double c )
```

6.10.2.7 constFunc() [3/4]

```
double constFunc (
    valarray< double > & point,
    const vector< double > & param )
```

6.10.2.8 constFunc() [4/4]

```
double constFunc (
    valarray< double > & point,
    const vector< double > & param,
    double c )
```

6.10.2.9 delete_pointed_to()

```
template<typename T >
void delete_pointed_to (
    T const ptr )
```

6.10.2.10 factorial() [1/2]

```
int factorial (
    int top )
```

6.10.2.11 factorial() [2/2]

```
int factorial (
    int top,
    int bottom )
```

6.10.2.12 FlatIndic() [1/4]

```
int FlatIndic (
    const int M,
    const indice & p )
```


6.10.2.13 FlatIndic() [2/4]

```
int FlatIndic (
    const int M,
    int i,
    int j )
```

6.10.2.14 FlatIndic() [3/4]

```
int FlatIndic (
    const MeshInfo & mi,
    const indice & p )
```

6.10.2.15 FlatIndic() [4/4]

```
int FlatIndic (
    const MeshInfo & mi,
    int i,
    int j )
```

6.10.2.16 MPIGlobalToLocal()

```
indice MPIGlobalToLocal (
    indice global,
    const MeshInfo & mi )
```

6.10.2.17 MPILocalToGlobal()

```
indice MPILocalToGlobal (
    indice local,
    const MeshInfo & mi )
```

6.10.2.18 polyEval()

```
double polyEval (
    double x,
    double * coef,
    int degree )
```

6.10.2.19 polynDerMulti()

```
void polynDerMulti (
    int der,
    int max,
    int * multiplier )
```

6.11 util.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UTIL_H_
00002 #define UTIL_H_
00003
00004 #include <vector>
00005 #include <utility>
00006 #include <set>
00007 #include <unordered_set>
00008 #include <array>
00009 #include <valarray>
00010 #include <algorithm>
00011 #include <numeric>
00012 #include <memory>
00013 #include <cstdlib>
00014 #include <type_traits>
00015 #include <iomanip>
00016
00017 #include "lapacke.h"
00018 #include "integral.h"
00019 #include <assert.h>
00020
00021 #include <petsc.h>
00022
00023 using vertex = valarray<double>;
00024 using indice = valarray<int>;
00025
00026 using vertexSet = vector<valarray<double>>;
00027 using indiceSet = vector<valarray<double>>;
00028
00029 template <typename T>
00030 void delete_pointed_to(T const ptr){
00031     delete ptr;
00032 }
00033
00034 /*
00035  *Containing essential information about mesh.
00036  */
00037 typedef struct {
00038
00041
00042     int dim;
00043
00044     vector<int> MPIlocalCellSize;
00045     vector<int> MPIlocalVertexSize;
00046
00047     indice MPIlocalCellStart;
00048     indice MPIlocalVertexStart;
00049
00050     vector<int> MPIglobalCellSize;
00051     vector<int> MPIglobalVertexSize;
00052
00053     int cellGhostLayerSize;
00054     int vertexGhostLayerSize;
00055
00056     vector<int> MPIlocalCellSizeFull;
00057     vector<int> MPIlocalVertexSizeFull;
00058
00059     // Horizontal edges first than vertical edges
00060     int MPIglobalHoriEdgeSize;
00061     int MPIlocalHoriEdgeSize;
00062
00063     // Starting at 0 (has not stacked on horizontal edges yet)
00064     int MPIglobalVertEdgeSize;
00065     int MPIlocalVertEdgeSize;
00066
00067     // Containing all the local mesh vertex points here
00068     vertexSet lmesh;
00069
00070     double** localVals;
```

```

00071
00072 // Corner index within a single element
00073 const indiceSet edgeCorner    {{0}, {1}};
00074 const indiceSet faceCorner    {{0,0},{1,0},{1,1},{0,1}};
00075 const indiceSet volumeCorner  {{0,0,0},{1,0,0},{1,1,0},{0,1,0},
00076                                {0,0,1},{1,0,1},{1,1,1},{0,1,1}};
00077 } MeshInfo;
00078
00079 void AssignValuesMeshInfo(MeshInfo& mi, DM dmv, DM dmu);
00080
00081 // Generic auxiliary functions
00082 // Function return constant value
00083 double constFunc(valarray<double>& point, const vector<double>& param);
00084
00085 double constFunc(valarray<double>& point, const vector<double>& param, double c);
00086
00087 double constFunc();
00088
00089 double constFunc(double c);
00090
00091 // Functions calculate factorials
00092 int factorial(int top, int bottom);
00093
00094 int factorial(int top);
00095
00096 double basePoly(vertex& point, const vector<int>& param);
00097
00098 // Evaluation of polynomial using Horner's method
00099 double polyEval(double x, double * coef, int degree);
00100
00101 // Compute factorial coefficient for polynomial derivatives
00102 void polynDerMulti(int der, int max, int * multiplier);
00103
00104 // Local to global and global to local
00105 // All indices are referenced to cell indice
00106 indice MPILocalToGlobal(indice local, const MeshInfo& mi);
00107 indice MPIGlobalToLocal(indice global, const MeshInfo& mi);
00108
00109 // Indice convention functions
00110 // Flatten indice into 1D array
00111 int FlatIndic(const MeshInfo& mi, int i, int j);
00112
00113 int FlatIndic(const int M, int i, int j);
00114 int FlatIndic(const MeshInfo& mi, const indice& p);
00115 int FlatIndic(const int M, const indice& p);
00116
00117 // Reverse process of flatten indices
00118 indice Bend(const MeshInfo& mi, int flat);
00119
00120 indice Bend(const int M, int flat);
00121
00122 #endif

```


Index

[/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp](#), 35
[/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h](#), 35, 36
[/home/renpo/Research/MantleSolver/src/mlweno/reconstruction.cpp](#), 37
[/home/renpo/Research/MantleSolver/src/mlweno/reconstruction.h](#), 37, 38
[/home/renpo/Research/MantleSolver/src/mlweno/stencil.h](#), 39, 40
[/home/renpo/Research/MantleSolver/src/mlweno/util.cpp](#), 41
[/home/renpo/Research/MantleSolver/src/mlweno/util.h](#), 45, 50

`~basePolynomial`
 [tensorProductPoly::basePolynomial](#), 10
`~multiLevelReconstruction`
 [MLWENO::multiLevelReconstruction](#), 18
`~singleLevelReconstruction`
 [MLWENO::singleLevelReconstruction](#), 22
`~stencil`
 [stencil< T >](#), 26
`~stencilPolynomial`
 [tensorProductPoly::stencilPolynomial](#), 30

`AddLevel`
 [MLWENO::multiLevelReconstruction](#), 18
`AssignValuesMeshInfo`
 [util.cpp](#), 42
 [util.h](#), 46

`basePoly`
 [util.cpp](#), 42
 [util.h](#), 47
`basePolynomial`
 [tensorProductPoly::basePolynomial](#), 9, 10
`Bend`
 [util.cpp](#), 42
 [util.h](#), 47

`CalculateSmoothnessIndic`
 [MLWENO::singleLevelReconstruction](#), 22
`cellCentered`
 [stencil.h](#), 40
`cellGhostLayerSize`
 [MeshInfo](#), 13
`CheckExist`
 [MLWENO::singleLevelReconstruction](#), 22
`CheckStencilPolynomials`
 [MLWENO::singleLevelReconstruction](#), 22

`CheckStencils`
 [MLWENO::singleLevelReconstruction](#), 23
`Clear`
 [MLWENO::multiLevelReconstruction](#), 19
`clearRcpp`
 [stencil< T >](#), 26
`coeffFunc`
 [util.cpp](#), 42, 43
 [util.h](#), 47, 48
`CreateStencil`
 [stencil< T >](#), 26
`CreateStencilPolynomials`
 [MLWENO::singleLevelReconstruction](#), 23

`delete_pointed_to`
 [util.h](#), 48
`der`
 [tensorProductPoly::basePolynomial](#), 10
`dim`
 [MeshInfo](#), 13

`edgeCentered`
 [stencil.h](#), 40
`edgeCorner`
 [MeshInfo](#), 13
`eval`
 [tensorProductPoly::basePolynomial](#), 10
 [tensorProductPoly::stencilPolynomial](#), 31

`faceCorner`
 [MeshInfo](#), 14
`factorial`
 [util.cpp](#), 43
 [util.h](#), 48
`FlatIndic`
 [util.cpp](#), 43, 44
 [util.h](#), 48, 49

`getCoef`
 [tensorProductPoly::basePolynomial](#), 11
`getI`
 [stencil< T >](#), 26
 [tensorProductPoly::basePolynomial](#), 11
`GetInfo`
 [MLWENO::multiLevelReconstruction](#), 19
`getJ`
 [stencil< T >](#), 26
 [tensorProductPoly::basePolynomial](#), 11
`getK`
 [stencil< T >](#), 27

- GetOrderX
 - tensorProductPoly::stencilPolynomial, 31
- GetOrderY
 - tensorProductPoly::stencilPolynomial, 31
- GetScale
 - MLWENO::singleLevelReconstruction, 23
 - tensorProductPoly::stencilPolynomial, 31
- getSize
 - stencil < T >, 27
 - tensorProductPoly::basePolynomial, 11
- GetSizeX
 - MLWENO::singleLevelReconstruction, 23
- GetSizeY
 - MLWENO::singleLevelReconstruction, 23
- GetSmoothIndic
 - tensorProductPoly::stencilPolynomial, 31
- GetSmoothnessIndic
 - MLWENO::singleLevelReconstruction, 24
- GetStencil
 - stencil < T >, 27
- indice
 - util.h, 46
- indiceSet
 - util.h, 46
- Imesh
 - MeshInfo, 14
- localVals
 - MeshInfo, 14
- MeshInfo, 12
 - cellGhostLayerSize, 13
 - dim, 13
 - edgeCorner, 13
 - faceCorner, 14
 - Imesh, 14
 - localVals, 14
 - MPIglobalCellSize, 14
 - MPIglobalHoriEdgeSize, 14
 - MPIglobalVertEdgeSize, 14
 - MPIglobalVertexSize, 14
 - MPIlocalCellSize, 15
 - MPIlocalCellSizeFull, 15
 - MPIlocalCellStart, 15
 - MPIlocalHoriEdgeSize, 15
 - MPIlocalVertEdgeSize, 15
 - MPIlocalVertexSize, 15
 - MPIlocalVertexSizeFull, 16
 - MPIlocalVertexStart, 16
 - vertexGhostLayerSize, 16
 - volumeCorner, 16
- MLWENO, 7
- MLWENO::multiLevelReconstruction, 16
 - ~multiLevelReconstruction, 18
 - AddLevel, 18
 - Clear, 19
 - GetInfo, 19
 - ModifyReconstMethod, 19
 - multiLevelReconstruction, 17
 - PrintBoundaryLayer, 19
 - PrintNonLinearWgts, 20
 - PrintSmoothnessIndicator, 20
 - SelectWenoReconstLevel, 20
 - SeparateBoundaryLayer, 20
 - UpdateNonLinearWgts, 20
- MLWENO::singleLevelReconstruction, 21
 - ~singleLevelReconstruction, 22
 - CalculateSmoothnessIndic, 22
 - CheckExist, 22
 - CheckStencilPolynomials, 22
 - CheckStencils, 23
 - CreateStencilPolynomials, 23
 - GetScale, 23
 - GetSizeX, 23
 - GetSizeY, 23
 - GetSmoothnessIndic, 24
 - PrintSmoothnessIndicator, 24
 - singleLevelReconstruction, 21, 22
 - UpdateSmoothnessIndic, 24
- ModifyReconstMethod
 - MLWENO::multiLevelReconstruction, 19
- MPIglobalCellSize
 - MeshInfo, 14
- MPIglobalHoriEdgeSize
 - MeshInfo, 14
- MPIGlobalToLocal
 - util.cpp, 44
 - util.h, 49
- MPIglobalVertEdgeSize
 - MeshInfo, 14
- MPIglobalVertexSize
 - MeshInfo, 14
- MPIlocalCellSize
 - MeshInfo, 15
- MPIlocalCellSizeFull
 - MeshInfo, 15
- MPIlocalCellStart
 - MeshInfo, 15
- MPIlocalHoriEdgeSize
 - MeshInfo, 15
- MPILocalToGlobal
 - util.cpp, 44
 - util.h, 49
- MPIlocalVertEdgeSize
 - MeshInfo, 15
- MPIlocalVertexSize
 - MeshInfo, 15
- MPIlocalVertexSizeFull
 - MeshInfo, 16
- MPIlocalVertexStart
 - MeshInfo, 16
- multiLevelReconstruction
 - MLWENO::multiLevelReconstruction, 17
- operator()
 - stencil < T >, 27, 28
 - tensorProductPoly::basePolynomial, 11

- tensorProductPoly::stencilPolynomial, [31](#), [32](#)
- polyEval
 - util.cpp, [44](#)
 - util.h, [49](#)
- polynDerMulti
 - util.cpp, [44](#)
 - util.h, [49](#)
- PrintBoundaryLayer
 - MLWENO::multiLevelReconstruction, [19](#)
- printCoef
 - tensorProductPoly::basePolynomial, [12](#)
 - tensorProductPoly::stencilPolynomial, [32](#)
- PrintNonLinearWgts
 - MLWENO::multiLevelReconstruction, [20](#)
- PrintSmoothnessIndicator
 - MLWENO::multiLevelReconstruction, [20](#)
 - MLWENO::singleLevelReconstruction, [24](#)
- SelectWenoReconstLevel
 - MLWENO::multiLevelReconstruction, [20](#)
- SeparateBoundaryLayer
 - MLWENO::multiLevelReconstruction, [20](#)
- setCoef
 - tensorProductPoly::basePolynomial, [12](#)
- SetCollapsePolyn
 - tensorProductPoly::stencilPolynomial, [32](#)
- setI
 - stencil < T >, [28](#)
- setJ
 - stencil < T >, [28](#)
- setK
 - stencil < T >, [29](#)
- setMaxDegree
 - tensorProductPoly::basePolynomial, [12](#)
- SetStencil
 - stencil < T >, [29](#)
- SetStencilPolynomials
 - tensorProductPoly::stencilPolynomial, [32](#)
- SetUpScale
 - tensorProductPoly::stencilPolynomial, [32](#), [33](#)
- singleLevelReconstruction
 - MLWENO::singleLevelReconstruction, [21](#), [22](#)
- stencil
 - stencil < T >, [25](#), [26](#)
- stencil < T >, [24](#)
 - ~stencil, [26](#)
 - clearPtr, [26](#)
 - CreateStencil, [26](#)
 - getI, [26](#)
 - getJ, [26](#)
 - getK, [27](#)
 - getSize, [27](#)
 - GetStencil, [27](#)
 - operator(), [27](#), [28](#)
 - setI, [28](#)
 - setJ, [28](#)
 - setK, [29](#)
 - SetStencil, [29](#)
 - stencil, [25](#), [26](#)
- stencil.h
 - cellCentered, [40](#)
 - edgeCentered, [40](#)
 - stencilType, [40](#)
 - vertexCentered, [40](#)
- stencilPolynomial
 - tensorProductPoly::stencilPolynomial, [30](#)
- stencilType
 - stencil.h, [40](#)
- tensorProductPoly, [7](#)
- tensorProductPoly::basePolynomial, [9](#)
 - ~basePolynomial, [10](#)
 - basePolynomial, [9](#), [10](#)
 - der, [10](#)
 - eval, [10](#)
 - getCoef, [11](#)
 - getI, [11](#)
 - getJ, [11](#)
 - getSize, [11](#)
 - operator(), [11](#)
 - printCoef, [12](#)
 - setCoef, [12](#)
 - setMaxDegree, [12](#)
- tensorProductPoly::stencilPolynomial, [29](#)
 - ~stencilPolynomial, [30](#)
 - eval, [31](#)
 - GetOrderX, [31](#)
 - GetOrderY, [31](#)
 - GetScale, [31](#)
 - GetSmoothIndic, [31](#)
 - operator(), [31](#), [32](#)
 - printCoef, [32](#)
 - SetCollapsePolyn, [32](#)
 - SetStencilPolynomials, [32](#)
 - SetUpScale, [32](#), [33](#)
 - stencilPolynomial, [30](#)
- UpdateNonLinearWgts
 - MLWENO::multiLevelReconstruction, [20](#)
- UpdateSmoothnessIndic
 - MLWENO::singleLevelReconstruction, [24](#)
- util.cpp
 - AssignValuesMeshInfo, [42](#)
 - basePoly, [42](#)
 - Bend, [42](#)
 - constFunc, [42](#), [43](#)
 - factorial, [43](#)
 - FlatIndic, [43](#), [44](#)
 - MPIGlobalToLocal, [44](#)
 - MPILocalToGlobal, [44](#)
 - polyEval, [44](#)
 - polynDerMulti, [44](#)
- util.h
 - AssignValuesMeshInfo, [46](#)
 - basePoly, [47](#)
 - Bend, [47](#)
 - constFunc, [47](#), [48](#)

- delete_pointed_to, [48](#)
- factorial, [48](#)
- FlatIndic, [48](#), [49](#)
- indice, [46](#)
- indiceSet, [46](#)
- MPIGlobalToLocal, [49](#)
- MPILocalToGlobal, [49](#)
- polyEval, [49](#)
- polynDerMulti, [49](#)
- vertex, [46](#)
- vertexSet, [46](#)

vertex

- util.h, [46](#)

vertexCentered

- stencil.h, [40](#)

vertexGhostLayerSize

- MeshInfo, [16](#)

vertexSet

- util.h, [46](#)

volumeCorner

- MeshInfo, [16](#)